AI-DRIVEN DEMAND FORECASTING & DYNAMIC PRICING IN RETAIL

AGENDA

- Introduction Problem
- Statement Dataset
- Overview
- Deep Learning Techniques
- Exploratory Data Analysis
- Model Development
- Model Evaluation
- Business Impact
- Conclusion

INTRODUCTION

- Retailers face rapidly changing customer demand across physical stores and e-commerce.
- Traditional demand forecasting and static pricing models struggle to keep up, leading to overstock, stockouts, and lost revenue.
- The project applies:
  - Temporal Fusion Transformer (TFT): for advanced, multi-step
  - demand forecasting. Deep Reinforcement Learning (DRL): for real-time dynamic pricing optimization.

PROJECT MOTIVATION

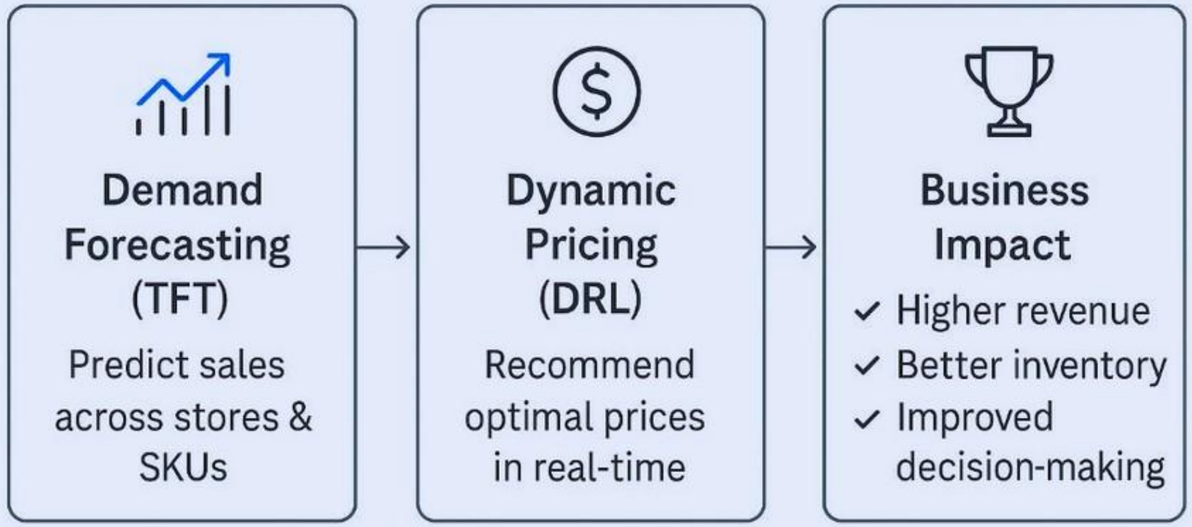
- Build a practical AI pipeline for demand forecasting and dynamic pricing in retail.
- Improve inventory allocation and minimize stock-related risks.
- Provide transparent, explainable AI to guide business decisions.
- Use the Walmart M5 Forecasting dataset to demonstrate real-world impact.

PROBLEM STATEMENT:

- Traditional forecasting and pricing models struggle in today's dynamic retail landscape.
- Aim to use deep learning to accurately forecast demand and optimize pricing across locations, reducing inventory issues and maximizing revenue.

BUSINESS QUESTIONS:

1. How can deep learning models accurately forecast multi-step product demand using sales, calendar events, and pricing data?
2. How can reinforcement learning dynamically optimize pricing strategies in real time to maximize revenue and profitability?



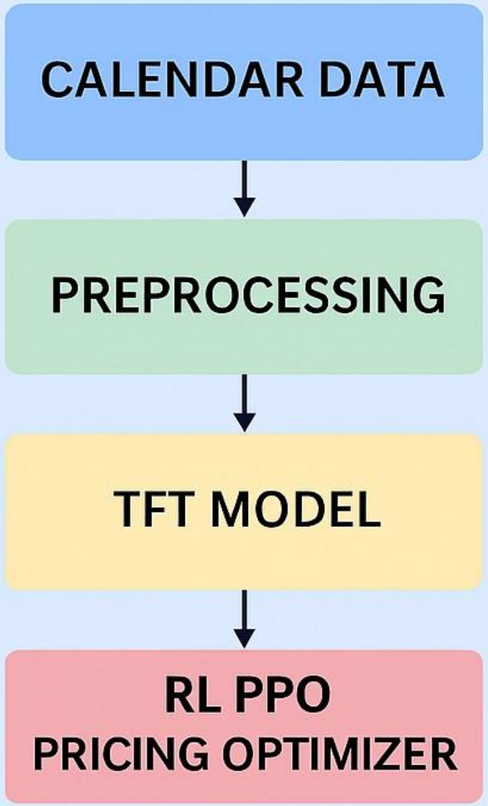
Dataset Details & Models Overview

Dataset: Walmart M5 Forecasting (Kaggle)	
Total SKUs	~3,000
Number of Stores	10
Historical Range	1.941 days (~5 years)
Data Points	Daily sales, selling prices
Hierarchy	Store → Department-SKU

Key Data Characteristics

- Time series type: Multi-level
- Main Variables: static: id
  + time-varying real: sales;
- Special Events: Holidays

Main Variable: id
+ time-varying: real
sales;
Special Events
SNAP (food sance days)



DATA LOADING

```python
# PyTorch & Lightning
import torch
import torch.nn as nn
from torch.optim import Adam
import pytorch_lightning as pl
from pytorch_lightning import Trainer
from pytorch_lightning.callbacks import EarlyStopping, LearningRateMonitor
from pytorch_lightning.loggers import TensorBoardLogger

# Pytorch Forecasting (TFT & utilities)
from pytorch_forecasting import TemporalFusionTransformer, TimeSeriesDataS

# Reinforcement Learning (Stable Baselines3)
from stable_baselines3 import PPO
from stable_baselines3.common.vec_env import DummyVecEnv
```

Crucial for implementing demand forecasting and dynamic pricing optimization

Dataset Loading

```python
# Load datasets
sales_df = pd.read_csv("sales_train_validation.csv")
calendar_df = pd.read_csv("calendar.csv")
prices_df = pd.read_csv( "sell_prices.csv")

# Check shapes
print(f"Sales data shape: {sales_df.shape}")
print(f"Calendar data shape: {calendar_df.shape}")
print(f"Prices data shape: {prices_df.shape}")
```

```
Sales data shape: (30490, 1919)
Calendar data shape: (1969, 14)
Prices data shape: (6841121, 4)
```

FILTERING AND PREPROCESSING

```python
# Filter by 1 store and top 30 item IDs
selected_store = 'CA_1'
selected_items = sales_df[sales_df['store_id'] == selected_store]['item_id'].unique()[:30]
sales_df_small = sales_df[
    (sales_df['store_id'] == selected_store) &
    (sales_df['item_id'].isin(selected_items))]

calendar_df['date'] = pd.to_datetime(calendar_df['date']) --------------------------> # Ensure date column is datetime

recent_dates = calendar_df['date'].sort_values().unique()[-180:]--------------------> # Get last N days of 'd' columns (e.g., last 180 days)
recent_days = calendar_df[calendar_df['date'].isin(recent_dates)]['d'].unique().tolist()

id_cols = ['id', 'item_id', 'dept_id', 'cat_id', 'store_id', 'state_id'] ------------> # Keep the sales columns for those days + identifier columns
available_days = [col for col in recent_days if col in sales_df_small.columns]
filtered_sales = sales_df_small[id_cols + available_days]

sales_long = pd.melt(filtered_sales,--------------------------------- --------------> # Melt the sales data to long format
    id_vars=['id', 'item_id', 'dept_id', 'cat_id', 'store_id', 'state_id'],
    var_name='d', value_name='sales')

sales_calendar = sales_long.merge(calendar_df, how='left', on='d') ------------------> # Merge with calendar and price data
full_data = sales_calendar.merge(prices_df, how='left', on=['store_id', 'item_id', 'wm_yr_wk'])
```

- **Choose data specifically for the store and the top 30 items.**
- **Change the date column to a datetime format.**
- **Filter for sales data from the last 180 days.**
- **Integrate sales, calendar, and pricing data for forecasting.**

```
Merged data shape: (3720, 22)
                          id        item_id    dept_id   cat_id store_id  \
0  HOBBIES_1_001_CA_1_validation  HOBBIES_1_001  HOBBIES_1  HOBBIES     CA_1
1  HOBBIES_1_002_CA_1_validation  HOBBIES_1_002  HOBBIES_1  HOBBIES     CA_1
2  HOBBIES_1_003_CA_1_validation  HOBBIES_1_003  HOBBIES_1  HOBBIES     CA_1
3  HOBBIES_1_004_CA_1_validation  HOBBIES_1_004  HOBBIES_1  HOBBIES     CA_1
4  HOBBIES_1_005_CA_1_validation  HOBBIES_1_005  HOBBIES_1  HOBBIES     CA_1

  state_id       d  sales       date  wm_yr_wk  ... month  year  event_name_1  \
0       CA  d_1790      3 2015-12-23     11547  ...    12  2015           NaN
1       CA  d_1790      3 2015-12-23     11547  ...    12  2015           NaN
2       CA  d_1790      2 2015-12-23     11547  ...    12  2015           NaN
3       CA  d_1790      0 2015-12-23     11547  ...    12  2015           NaN
4       CA  d_1790      3 2015-12-23     11547  ...    12  2015           NaN

   event_type_1 event_name_2 event_type_2 snap_CA snap_TX  snap_WI  sell_price
0           NaN          NaN          NaN       0       0        0        8.26
1           NaN          NaN          NaN       0       0        0        3.97
2           NaN          NaN          NaN       0       0        0        2.97
3           NaN          NaN          NaN       0       0        0        4.64
4           NaN          NaN          NaN       0       0        0        2.88

[5 rows x 22 columns]
```

fully merged dataset

FEATURE GENERATION & ENCODING

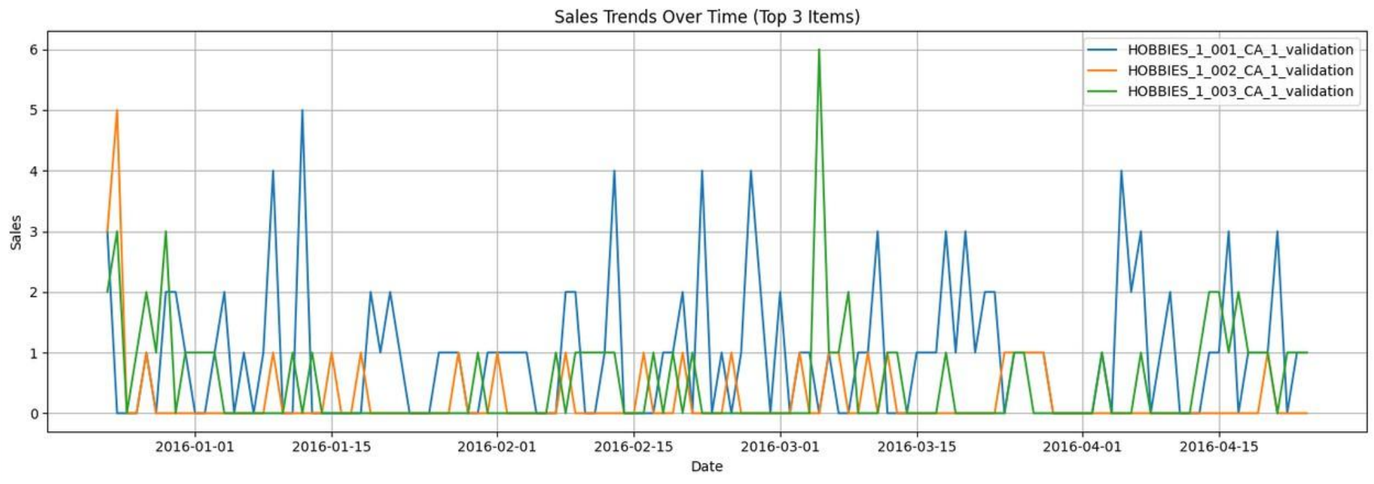
```python
import pandas as pd
from sklearn.preprocessing import LabelEncoder
# 1. Ensure 'date' is datetime
full_data['date'] = pd.to_datetime(full_data['date'])
# 2. Create a time index for TFT (days since first date)
full_data['time_idx'] = (full_data['date'] - full_data['date'].min()).dt.days
# 3. Sort for lag features
full_data = full_data.sort_values(by=['id', 'date'])
# 4. Create lag features
LAG_DAYS = [7]
for lag in LAG_DAYS:
    full_data[f'lag_{lag}'] = full_data.groupby('id')['sales'].shift(lag)
# 5. Create rolling mean features
ROLLING_WINDOWS = [7]
for window in ROLLING_WINDOWS:
    full_data[f'rolling_mean_{window}'] = (
        full_data.groupby('id')['sales']
        .shift(1)  # shift by 1 to avoid using current day's value
        .rolling(window)
        .mean()
        .reset_index(drop=True))
# 6. Create date-based features
full_data['day_of_week'] = full_data['date'].dt.dayofweek
full_data['week_of_year'] = full_data['date'].dt.isocalendar().week.astype('int16')
full_data['month'] = full_data['date'].dt.month
full_data['year'] = full_data['date'].dt.year
# Lag features
full_data["lag_1"] = full_data.groupby("id")["sales"].shift(1)
full_data["lag_7"] = full_data.groupby("id")["sales"].shift(7)
full_data["lag_14"] = full_data.groupby("id")["sales"].shift(14)
# Rolling means
full_data["rolling_mean_7"] = (
    full_data.groupby("id")["sales"].transform(lambda x: x.shift(1).rolling(7).mean()))
full_data["rolling_mean_14"] = (
    full_data.groupby("id")["sales"].transform(lambda x: x.shift(1).rolling(14).mean()))
# Extra time features
full_data["day_of_month"] = full_data["date"].dt.day
full_data["week_of_year"] = full_data["date"].dt.isocalendar().week.astype(int)
full_data["is_weekend"] = full_data["date"].dt.dayofweek.isin([5, 6]).astype(int)
# 7. Encode categorical features
cat_cols = ['item_id', 'dept_id', 'cat_id', 'store_id', 'state_id']
for col in cat_cols:
    full_data[col] = full_data[col].astype(str)  # Optionally: use LabelEncoder
# 8. Filter IDs with enough data history (>= 30 days)
min_required_days = 30
id_counts = full_data['id'].value_counts()
valid_ids = id_counts[id_counts > min_required_days].index
full_data = full_data[full_data['id'].isin(valid_ids)].copy()
# Print check
print(f"✅ Final feature-engineered shape: {full_data.shape}")
print(full_data[['sales', 'lag_7', 'rolling_mean_7', 'day_of_week', 'month']].head())
```

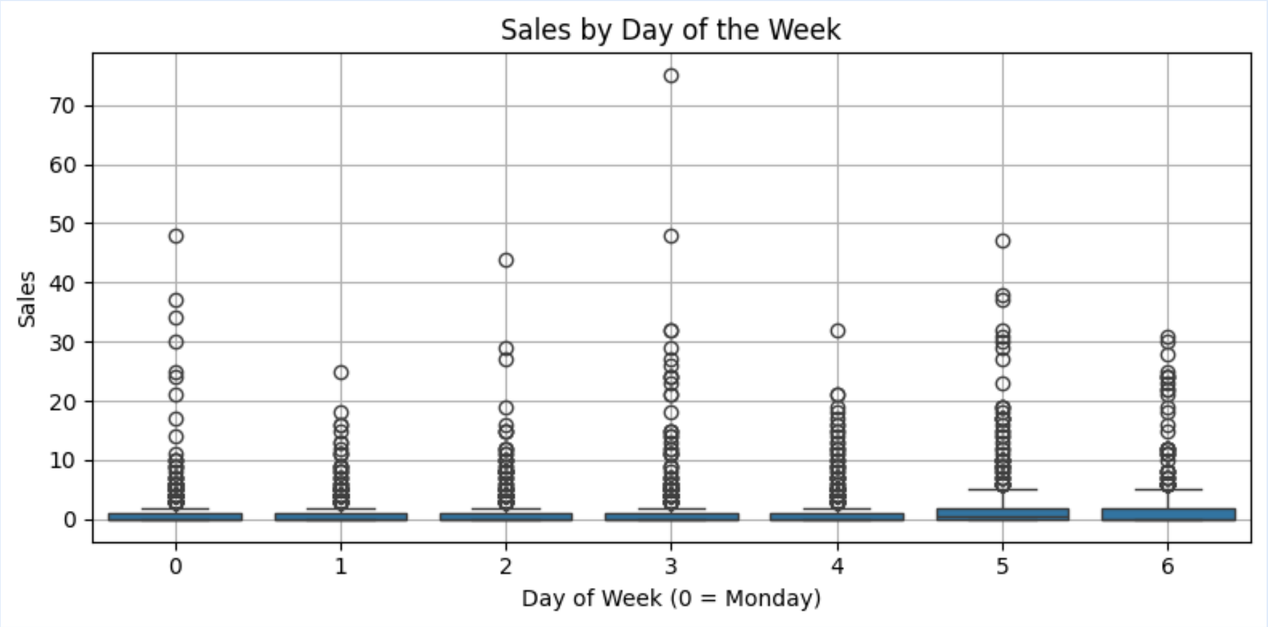
- Converted date to datetime format.
- Created time index for time series modeling.
- Generated lag features (lag_1, lag_7, lag_14) and rolling mean features (rolling_mean_7, rolling_mean_14) to capture temporal patterns and trends.
- Encoded categorical features like item_id.
- Dropped rows with NaN values.

```
✅ Final feature-engineered shape: (3314, 27)
   sales  lag_7  rolling_mean_7  day_of_week  month
0      2    3.0        0.142857            2     12
1      1    0.0        0.000000            3     12
2      0    0.0        0.285714            4      1
3      0    0.0        0.714286            5      1
4      1    1.0        0.714286            6      1
```

DATA EXPLORATION & VISUALIZATION

```python
# Ensure 'full_data' is already loaded
full_data["date"] = pd.to_datetime(full_data["date"])
full_data["day_of_week"] = full_data["date"].dt.dayofweek  # Monday=0
full_data["month"] = full_data["date"].dt.month

#1. Sales Trends for Sample Items
sample_items = full_data["id"].unique()[:3]
sample_df = full_data[full_data["id"].isin(sample_items)]

plt.figure(figsize=(14, 5))
for item in sample_items:
    item_df = sample_df[sample_df["id"] == item]
    plt.plot(item_df["date"], item_df["sales"], label=item)
plt.title("Sales Trends Over Time (Top 3 Items)")
plt.xlabel("Date")
plt.ylabel("Sales")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

#3. Boxplot of Sales by Day of Week
plt.figure(figsize=(8, 4))
sns.boxplot(x="day_of_week", y="sales", data=full_data)
plt.title("Sales by Day of the Week")
plt.xlabel("Day of Week (0 = Monday)")
plt.ylabel("Sales")
plt.grid(True)
plt.tight_layout()
plt.show()
```





DATASET CREATION & DATALOADER

```python
from lightning.pytorch import seed_everything
from pytorch_forecasting import TimeSeriesDataSet
from pytorch_forecasting.data import GroupNormalizer
seed_everything(42, workers=True)
required_cols = ["sales", "sell_price", "lag_1", "lag_7", "rolling_mean_7", "month", "year",
                 "day_of_week", "day_of_month", "week_of_year", "is_weekend",]
full_data = full_data.dropna(subset=required_cols).copy()
print("Cleaned full_data shape:", full_data.shape)
max_prediction_length = 7
training_cutoff = full_data["time_idx"].max() - max_prediction_length
training = TimeSeriesDataSet(
    full_data[lambda x: x.time_idx <= training_cutoff],
    time_idx="time_idx",
    target="sales",
    group_ids=["id"],
    max_encoder_length=30,
    max_prediction_length=max_prediction_length,
    static_categoricals=["item_id", "store_id", "dept_id", "cat_id", "state_id"],
    time_varying_known_reals=[ "time_idx", "month", "year", "day_of_month",
                              "week_of_year", "is_weekend",],
    time_varying_known_categoricals=["day_of_week"],
    time_varying_unknown_reals=[ "sales", "lag_1", "lag_7", "rolling_mean_7", "sell_price",],
    target_normalizer=GroupNormalizer(groups=["id"]),
    add_relative_time_idx=True,
    add_target_scales=True,
    add_encoder_length=True,
    allow_missing_timesteps=True,
)
validation = TimeSeriesDataSet.from_dataset(
    training, full_data, predict=True, stop_randomization=True
)
train_dataloader = training.to_dataloader(train=True, batch_size=64, num_workers=0)
val_dataloader = validation.to_dataloader(train=False, batch_size=64, num_workers=0)
print("✅ Training and validation dataloaders created successfully")
```

```
INFO: Seed set to 42
INFO:lightning.fabric.utilities.seed:Seed set to 42
Cleaned full_data shape: (3510, 32)
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: dat
  return datetime.utcnow().replace(tzinfo=utc)
✅ Training and validation dataloaders created successfully
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: dat
  return datetime.utcnow().replace(tzinfo=utc)
```

- **Created TimeSeriesDataSet for forecasting using features**
- **Used GroupNormalizer for target normalization to maintain consistent scaling.**
- **Removed rows with missing values in required forecasting inputs before constructing the TFT dataset.**
- **Defined a cutoff point for splitting data into training and validation sets using a temporal cutoff based on time_idx.**
- **Created train and validation DataLoaders for efficient data batching, with batch size set to 64.**

TFT MODEL TRAINING

```python
import lightning.pytorch as pl
from lightning.pytorch.callbacks import EarlyStopping, LearningRateMonitor, ModelCheckpoint
from lightning.pytorch.loggers import TensorBoardLogger
from pytorch_forecasting.models import TemporalFusionTransformer
from pytorch_forecasting.metrics import QuantileLoss
logger = TensorBoardLogger("lightning_logs", name="tft_m5_model")
early_stop_callback = EarlyStopping(
    monitor="val_loss",
    min_delta=1e-4,
    patience=4,
    verbose=True,
    mode="min")
lr_logger = LearningRateMonitor(logging_interval="epoch")
checkpoint_callback = ModelCheckpoint(monitor="val_loss", mode="min", save_top_k=1)
tft = TemporalFusionTransformer.from_dataset(
    training,
    learning_rate=0.03,
    hidden_size=16,
    attention_head_size=1,
    dropout=0.1,
    hidden_continuous_size=8,
    output_size=7,
    loss=QuantileLoss(),
    log_interval=10,
    reduce_on_plateau_patience=2)
trainer = pl.Trainer(
    max_epochs=30,
    accelerator="gpu" if torch.cuda.is_available() else "cpu",
    devices=1,
    gradient_clip_val=0.1,
    callbacks=[lr_logger, early_stop_callback, checkpoint_callback],
    logger=logger,
    enable_progress_bar=True)
trainer.fit(tft, train_dataloaders=train_dataloader, val_dataloaders=val_dataloader)
print("✅ TFT training complete.")
```

Logger & Early Stopping:
Used TensorBoardLogger and EarlyStopping for tracking and halting training

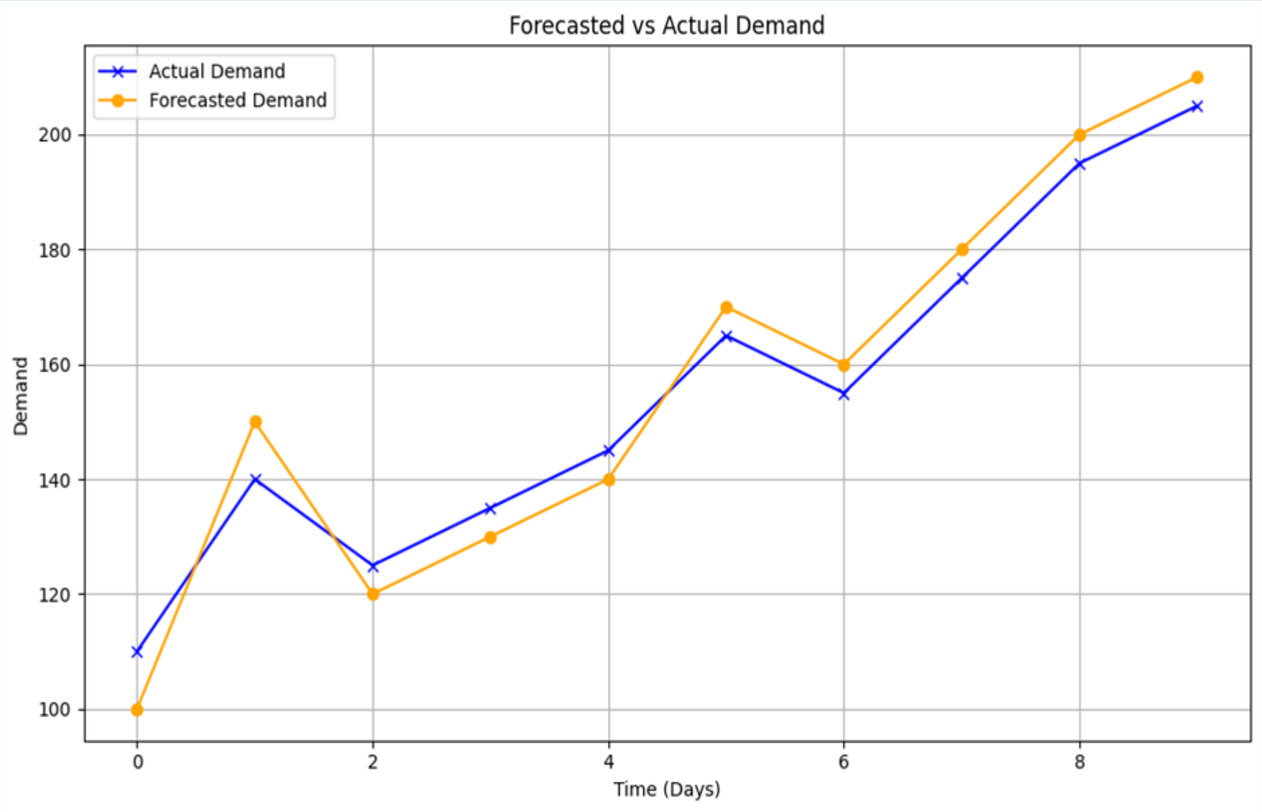
Temporal Fusion Transformer (TFT) Model:
Defined with learning rate 0.03, hidden_size 16, 7 output size, and QuantileLoss.

Training:
Configured for 30 epochs, with GPU support and gradient clipping at 0.1.

MODEL EVALUATION & FORECAST ACCURACY

```python
from pytorch_forecasting.metrics import SMAPE
import torch

best_model_path = trainer.checkpoint_callback.best_model_path
best_tft = TemporalFusionTransformer.load_from_checkpoint(best_model_path)

predictions = best_tft.predict(val_dataloader)
actuals = torch.cat([y[0] for x, y in iter(val_dataloader)])

min_len = min(predictions.shape[0], actuals.shape[0])
predictions = predictions[:min_len]
actuals = actuals[:min_len]

smape = SMAPE()(predictions, actuals)
print(f"✅ SMAPE on validation set: {smape:.3f}")
```

```
/usr/local/lib/python3.12/dist-packages/lightning/pytorch/utilities/parsing.py:213: 
/usr/local/lib/python3.12/dist-packages/lightning/pytorch/utilities/parsing.py:213: 
✅ SMAPE on validation set: 1.532
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWa
  return datetime.utcnow().replace(tzinfo=utc)
/usr/local/lib/python3.12/dist-packages/lightning/pytorch/utilities/_pytree.py:21: `
```



SMAPE = 1.532 (Moderate forecasting performance)

- **The model captures overall demand trends, but does not fully capture sharp fluctuations due to the sparse and highly variable nature of retail data.**

Forecast vs. Actual Demand plot:

- **The plot compares predicted and actual demand, showing that the model captures overall trends but struggles with sudden spikes and variability.**

DYNAMIC PRICING ENVIRONMENT (RL - PPO)

```python
class PricingEnv(gym.Env):
    def __init__(self, demand_data, cost_price=1.0, price_points=None, max_inventory=200):
        super(PricingEnv, self).__init__()
        self.demand_data = np.array(demand_data, dtype=np.float32)
        self.cost_price = cost_price
        self.price_points = price_points if price_points is not None else np.linspace(1.0, 10.0, 10)
        self.max_inventory = max_inventory

        self.action_space = spaces.Discrete(len(self.price_points))
        self.observation_space = spaces.Box(
            low=np.array([0, 0, 0], dtype=np.float32),
            high=np.array([self.max_inventory, np.max(self.demand_data), len(self.demand_data)], dtype=np.float32),
            dtype=np.float32 )
        self.reset()
    def reset(self):
        self.current_step = 0
        self.inventory = self.max_inventory
        return self._get_obs()
    def _get_obs(self):
        demand = self.demand_data[min(self.current_step, len(self.demand_data) - 1)]
        return np.array([
            self.inventory,
            demand,
            self.current_step
        ], dtype=np.float32)
    def step(self, action):
        price = self.price_points[int(action)]
        demand = self.demand_data[self.current_step]
        adjusted_demand = max(0, demand * (1 - 0.05 * (price - 1.0)))
        sold = min(adjusted_demand, self.inventory)
        revenue = price * sold
        cost = self.cost_price * sold
        inventory_penalty = 0.01 * self.inventory
        reward = revenue - cost - inventory_penalty

        self.inventory -= sold
        self.current_step += 1
        done = self.current_step >= len(self.demand_data) or self.inventory <= 0

        obs = self._get_obs() if not done else np.array([self.inventory, 0, self.current_step], dtype=np.float32)
        return obs, reward, done, {}
```

Defines a custom PricingEnv class for a dynamic pricing environment.

Initialises action space, observation space, and inventory constraints.

Resets the environment and steps through the time series with inventory and demand data.

Calculates revenue and updates inventory based on sales and pricing actions.

TRAINING PPO AGENT

```python
import gym
from gym import spaces
import numpy as np
from stable_baselines3 import PPO
from stable_baselines3.common.vec_env import DummyVecEnv, VecNormalize

# Sample demand data
sample_item_id = full_data['id'].value_counts().index[0]
sample_demand = full_data[full_data['id'] == sample_item_id]['sales'].values[:120]

# Initialize environment
env = PricingEnv(
    demand_data=sample_demand,
    cost_price=1.0,
    price_points=np.linspace(1.0, 10.0, 10),
    max_inventory=200
)

vec_env = DummyVecEnv([lambda: env])
vec_env = VecNormalize(vec_env, norm_obs=True, norm_reward=True, clip_obs=10.0)

ppo_agent = PPO("MlpPolicy", vec_env, verbose=1)
ppo_agent.learn(total_timesteps=10000)
```

Training Process:

- Environment is initialized with demand and pricing setup
- PPO agent predicts pricing actions at each timestep
- Rewards are calculated based on pricing decisions and demand response
- Agent iteratively updates its policy through interaction with the environment

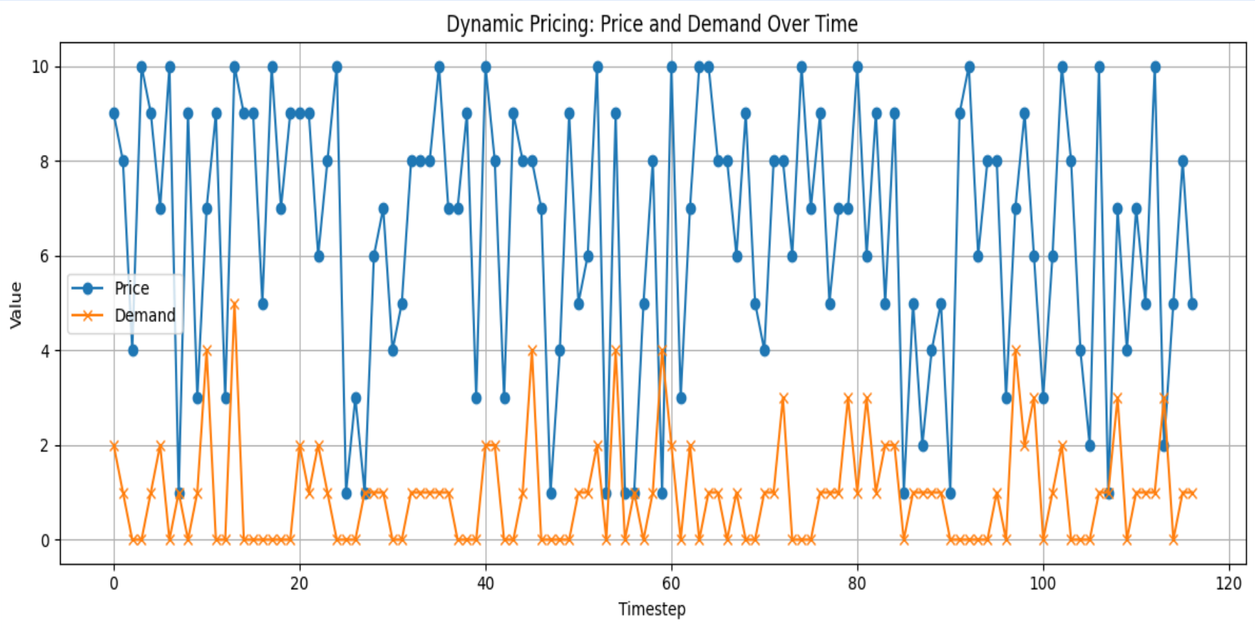
Training Observations:

- PPO agent trained for 10,000+ timesteps
- Explained variance improves during training
- Cumulative reward increases steadily
- Agent learns progressively better pricing strategies over time

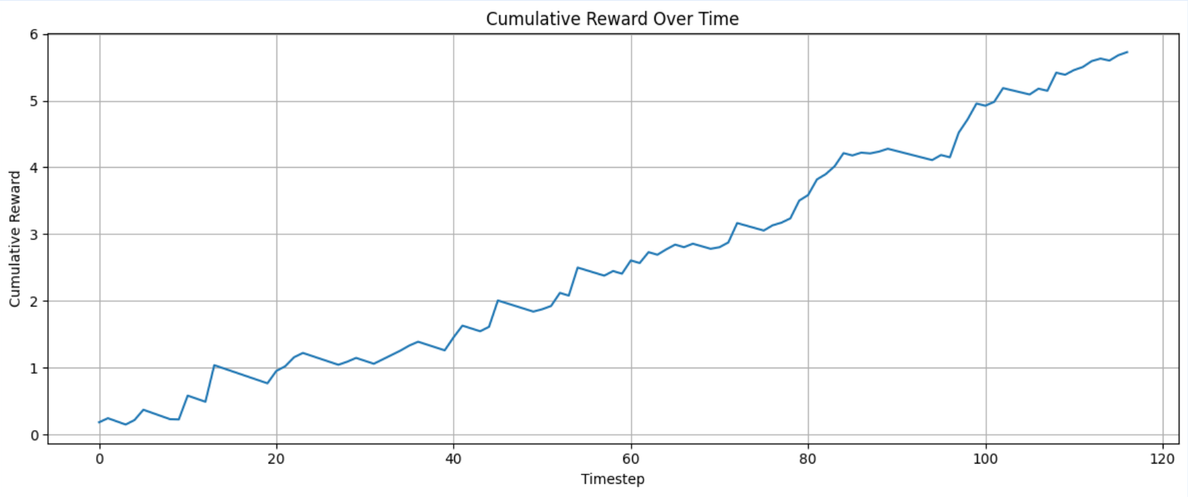
DYNAMIC PRICING OPTIMIZATION & EVALUATION

```python
obs = vec_env.reset()
total_rewards = []
prices = []
demands = []

for _ in range(len(sample_demand)):
    action, _states = ppo_agent.predict(obs)
    price = env.price_points[int(action[0])]
    prices.append(price)

    obs, reward, done, info = vec_env.step(action)
    total_rewards.append(float(reward[0]))

    demand = env.demand_data[min(env.current_step - 1, len(env.demand_data) - 1)]
    demands.append(float(demand))

    if done[0]:
        break
```

Reward Calculation: Calculates profit-based reward using price, cost, and demand.



- **Prices fluctuate dynamically over time, and demand responds to these pricing changes. The interaction between price and demand highlights how the agent explores different pricing strategies under varying demand conditions.**



- **Displays how cumulative reward (profit) changes over time as the model learns and improves.**

RECOMMENDATIONS FOR BUSINESS LEADERS

Demand Forecasting:
Adopt Temporal Fusion Transformers (TFT) to support multi-step product demand forecasting across time horizons, enhancing stock allocation and reducing lost sales.

Dynamic Pricing:
Implement Reinforcement Learning (PPO) models to optimize pricing in near real-time, increasing revenue and profit by responding to demand fluctuations.

Data-Driven Decisions:
Integrate calendar events, seasonality, and historical price data into forecasting & pricing pipelines to capture business cycles effectively.

Explainability & Trust:
Use attention weights & SHAP explainability to ensure the AI system is interpretable, providing confidence to leadership and stakeholders.

FUTURE SCOPE AND CONCLUSION

BROADER APPLICATION

Scale the solution to cover more product categories, locations, and integrate competitor data for a competitive edge.

ADVANCED ENHANCEMENTS

Transition to real-time systems and expand to include promotion and discount optimization for holistic retail intelligence.

KEY TAKEAWAY

This project demonstrates an integrated AI pipeline combining demand forecasting and dynamic pricing. It highlights how deep learning and reinforcement learning can support data-driven retail decision-making.

THANK YOU